

DD2440 Advanced Algorithms
Fall 2026
Lecture 5: Fast Fourier Transform

Ioana-Oriana Bercea

2026-10-01

Agenda

Polynomial multiplication

Fast Fourier Transform

Reminders

- ▶ individual assignment 1 released this Friday at 8:00

Agenda

Polynomial multiplication

Fast Fourier Transform

Polynomials

e.g., $A(x) = x^2 + 1$, $B(x) = 2x + 7$

General: $P(x) = a_0 + a_1 \cdot x + \dots + a_n \cdot x^n = \sum_{i=0}^n a_i \cdot x^i$

- ▶ the a_i 's are known as the *coefficients* of $P(x)$
- ▶ NB: can define a_i for all $i = 0, 1, \dots$
- ▶ *degree* of a polynomial: the highest degree of an individual term with non-zero coefficient
 - ▶ if $a_n \neq 0$ and $a_i = 0$ for all $i > n$, then degree is n
 - ▶ NB: a degree n polynomial has $n + 1$ non-zero coefficients

Operations on polynomials

- ▶ *evaluation*: given a specific number c , what is the number $P(c)$?
- ▶ *addition*: given P and Q , compute a polynomial $R = P + Q$ so that $R(x) = P(x) + Q(x)$ for all x
 - ▶ if P and Q both have degree n , then R will have degree **at most** n
- ▶ *multiplication*: given P and Q , compute a polynomial $R = P \cdot Q$ so that $R(x) = P(x) \cdot Q(x)$ for all x
 - ▶ if P and Q both have degree n , then R will have degree $2n$

Implementations

All the operations can be implemented:

- ▶ but it depends on how you actually **represent** the polynomial in the code
- ▶ each representation comes with trade-offs: some operation might be fast, but another will be slow
- ▶ we will discuss *two representations* today
 - ▶ coefficient representation (classic): an array of numbers a_i
such that:
$$P(x) = \sum_{i=0}^n a_i \cdot x^i$$
 - ▶ point-value representation (perhaps new) : pairs of numbers (x_i, y_i) , where $y_j = P(x_j)$

Interlude

Why are these operations useful?

1. Describe an algorithm that determines whether a given set of n integers contains two elements whose sum is zero, in $O(n \log n)$ time.
2. Describe an algorithm that determines whether a given set of n integers contains three elements whose sum is zero, in $O(n^2)$ time.
3. Now suppose the input set X contains only integers between $-10000n$ and $10000n$. Describe an algorithm that determines whether X contains three elements whose sum is zero, in $O(n \log n)$ time.

Implementations in coefficient representation (1)

Given: $P(x) = a_0 + a_1 \cdot x + \dots + a_n \cdot x^n = \sum_{j=0}^n a_j \cdot x^j$

- ▶ can represent it as an array P , where $P[j] = a_j$ for $j = 0, \dots, n$

Evaluation:

- ▶ $O(n)$ arithmetic operations

EVALUATE($P[0..n], x$):

$X \leftarrow 1$ $\langle\langle X = x^j \rangle\rangle$

$y \leftarrow 0$

for $j \leftarrow 0$ to n

$y \leftarrow y + P[j] \cdot X$

$X \leftarrow X \cdot x$

return y

Implementations in coefficient representation (1)

Given: $P(x) = a_0 + a_1 \cdot x + \dots + a_n \cdot x^n = \sum_{j=0}^n a_j \cdot x^j$

- ▶ can represent it as an array P , where $P[j] = a_j$ for $j = 0, \dots, n$

Evaluation:

- ▶ $O(n)$ arithmetic operations
- ▶ or reduce the number of multiplications by using Horner's rule: $P(x) = a_0 + x \cdot (a_1 + x \cdot (a_2 + \dots))$

EVALUATE($P[0..n], x$):

```
 $X \leftarrow 1$      $\langle\langle X = x^j \rangle\rangle$   
 $y \leftarrow 0$   
for  $j \leftarrow 0$  to  $n$   
     $y \leftarrow y + P[j] \cdot X$   
     $X \leftarrow X \cdot x$   
return  $y$ 
```

HORNER($P[0..n], x$):

```
 $y \leftarrow P[n]$   
for  $i \leftarrow n-1$  down to  $0$   
     $y \leftarrow x \cdot y + P[i]$   
return  $y$ 
```

Implementations in coefficient representation (2)

Given: $P(x) = \sum_{i=0}^n a_i \cdot x^i$ and $Q(x) = \sum_{i=0}^n b_i \cdot x^i$ (two arrays)

Addition: output $R = P + Q$, $R(x) = \sum_{i=0}^n (a_i + b_i) \cdot x^i$

► $O(n)$ arithmetic operations

<u>ADD($P[0..n], Q[0..n]$):</u> for $j \leftarrow 0$ to n $R[j] \leftarrow P[j] + Q[j]$ return $R[0..n]$
--

Implementations in coefficient representation (3)

Given: $P(x) = \sum_{j=0}^n a_j \cdot x^j$ and $Q(x) = \sum_{j=0}^n b_j \cdot x^j$ (two arrays)

Multiplication: output $R = P \cdot Q$

$$R(x) = \left(\sum_{j=0}^n a_j \cdot x^j \right) \cdot \left(\sum_{j=0}^n b_j \cdot x^j \right)$$

► $O(n^2)$ arithmetic operations

MULTIPLY($P[0..n], Q[0..m]$):

 for $j \leftarrow 0$ to $n + m$

$R[j] \leftarrow 0$

 for $j \leftarrow 0$ to n

 for $k \leftarrow 0$ to m

$R[j + k] \leftarrow R[j + k] + P[j] \cdot Q[k]$

 return $R[0..n + m]$

Polynomial multiplication as convolution

$$R(x) = \left(\sum_{j=0}^n a_j \cdot x^j \right) \cdot \left(\sum_{j=0}^n b_j \cdot x^j \right)$$

- ▶ can also be written as

$$R(x) = \sum_{k=0}^{2n-2} c_k \cdot x^k \text{ where } c_k = \sum_{i=\max\{0, k-n+1\}}^{\min\{k, n-1\}} a_i \cdot b_{k-i}$$

- ▶ this is the same as *(discrete) vector convolution*

- ▶ discrete $f * g[k] = \sum_{i=-\infty}^{\infty} f[i] \cdot g[k-i]$

- ▶ convolution is a more general term
 - ▶ it's **EVERYWHERE**: signal processing, CNN etc.

Point-value representation

Fundamental Theorem of Algebra: given $n + 1$ pairs $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, there is a **unique** polynomial P of **degree n** such that $P(x_i) = y_i$ for all $i \in \{0, \dots, n\}$.

- ▶ you say that P *interpolates* the points (x_i, y_i)
- ▶ so you can represent P of degree n by evaluating it at **any** $n + 1$ different numbers $x_0, \dots, x_n$
- ▶ then you specify P by giving **a list of pairs** $(x_0, P(x_0), \dots, (x_n, P(x_n)))$
- ▶ think of them as $n + 1$ *samples*

Implementations in point-value representation (1)

Adding and multiplying now are *easy*:

- ▶ both take $O(n)$ time
- ▶ addition
 - ▶ as long as P and Q have the same samples
 - ▶ set $R(x_i) = P(x_i) + Q(x_i)$ for every $i \in \{0, \dots, n\}$
- ▶ multiplication:
 - ▶ need $2n + 1$ samples to begin with, for both P and Q
 - ▶ set $R(x_i) = P(x_i) \cdot Q(x_i)$ for every $i \in \{0, \dots, 2n\}$

Implementations in point-value representation (2)

Evaluation is slightly trickier:

- ▶ takes $O(n^2)$ time
- ▶ uses Lagrange's formula

$$p(x) = \sum_{i=0}^n y_i \cdot \prod_{k \neq i} \frac{x - x_k}{x_i - x_k}$$

- ▶ intuition: think of the formula for a line going through two points (x_1, y_1) and (x_2, y_2)

Implementations vs representations

Representation	Evaluate	Add	Multiply
Coefficients	$O(n)$	$O(n)$	$O(n^2)$
Point-value	$O(n^2)$	$O(n)$	$O(n)$

- ▶ trade-offs
- ▶ ideal: quickly switch between representations

Implementations vs representations

Representation	Evaluate	Add	Multiply
Coefficients	$O(n)$	$O(n)$	$O(n^2)$
Point-value	$O(n^2)$	$O(n)$	$O(n)$

- ▶ trade-offs
- ▶ ideal: quickly switch between representations

TODAY: The Fast Fourier Transform is a way to convert between these two representations in $O(n \log n)$ time.

A general framework for conversion

Coefficients $\rightarrow$ point-value: $O(n^2)$ in total

- ▶ given coefficients a_j
- ▶ for each sample point x_i , evaluate the polynomial at $p(x_i)$
- ▶ $O(n)$ time per sample point

Point-value $\rightarrow$ coefficients: $O(n^3)$ in total

- ▶ given samples (x_i, y_i) for P
- ▶ to get a_0 : evaluate at $x = 0$, takes $O(n^2)$ time
- ▶ to get a_1 : get new polynomial $P_1(x) = (P(x) - a_0)/x$ ("reverse" Horner)
 - ▶ set $y_i = y_i - a_0$, throw out the sample $(0, 0)$
 - ▶ $a_1 =$ evaluate P_1 at 0, takes $O(n^2)$
- ▶ $O(n^2)$ time per coefficient

A different way

- ▶ for simplicity: assume the polynomial has degree $n - 1$
- ▶ so it has n coefficients $\{a_j\}$ and n point-value pairs $\{(x_i, y_i)\}$
- ▶ now define the vectors $\vec{a} = (a_0, a_1, \dots, a_{n-1})$, $\vec{y} = (y_0, y_1, \dots, y_{n-1})$ and the *Vandermonde* matrix:

$$V = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{bmatrix}.$$

- ▶ note that $V[i][j] = (x_i)^j$, then:

$$V \cdot \vec{a} = \vec{y}$$

A different way (2)

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{n-1} \end{bmatrix}$$

- ▶ Coefficients $\rightarrow$ point-value: $O(n^2)$ time
- ▶ Point-value $\rightarrow$ coefficients
 - ▶ solve a system of n linear equations in n unknowns (Gaussian elimination): $O(n^3)$ time
 - ▶ Matrix inverse, $\vec{a} = V^{-1} \cdot \vec{y}$: $O(n^2)$ time if you hardcode V^{-1}

A different way (2)

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{n-1} \end{bmatrix}$$

- ▶ Coefficients $\rightarrow$ point-value: $O(n^2)$ time
- ▶ Point-value $\rightarrow$ coefficients
 - ▶ solve a system of n linear equations in n unknowns (Gaussian elimination): $O(n^3)$ time
 - ▶ Matrix inverse, $\vec{a} = V^{-1} \cdot \vec{y}$: $O(n^2)$ time if you hardcode V^{-1}

Key idea: pick the sample points $x_0, x_1, \dots, x_{n-1}$ (and so the matrix V) in a clever way so that both steps require $O(n \log n)$ time.

Interlude

Why are these operations useful?

1. Describe an algorithm that determines whether a given set of n integers contains two elements whose sum is zero, in $O(n \log n)$ time.
2. Describe an algorithm that determines whether a given set of n integers contains three elements whose sum is zero, in $O(n^2)$ time.
3. Now suppose the input set X contains only integers between $-10000n$ and $10000n$. Describe an algorithm that determines whether X contains three elements whose sum is zero, in $O(n \log n)$ time.

Agenda

Polynomial multiplication

Fast Fourier Transform

The goal

So we want to choose $x_0, x_1, \dots, x_{n-1}$ so that:

- ▶ we get $O(n \log n)$ conversion from coefficients $\rightarrow$ point-value using faster evaluations
- ▶ we get $O(n \log n)$ conversion from point-value $\rightarrow$ coefficients using $\vec{a} = V^{-1} \cdot \vec{y}$
- ▶ use **divide and conquer**, i.e.,
 - ▶ break up the problem on n samples into 2 subproblems on $n/2$ samples
 - ▶ recurse
 - ▶ stitch them back together

Divide and conquer

Start with conversion from coefficients $\rightarrow$ point-value

OBS: $P(x) = P_{\text{even}}(x^2) + x \cdot P_{\text{odd}}(x^2)$

- ▶ the coefficients of P_{even} are the even-degree coefficients of P
 $P_{\text{even}}(x^2) = a_0 + a_2 \cdot x^2 + a_4 \cdot x^4 \dots$
- ▶ the coefficients of P_{odd} are the odd-degree coefficients of P
 $P_{\text{odd}}(x^2) = a_1 + a_3 \cdot x^2 + a_5 \cdot x^4 \dots$
- ▶ both P_{even} and P_{odd} have $\lceil n/2 \rceil$ coefficients

Divide and conquer

Start with conversion from coefficients $\rightarrow$ point-value

OBS: $P(x) = P_{\text{even}}(x^2) + x \cdot P_{\text{odd}}(x^2)$

- ▶ the coefficients of P_{even} are the even-degree coefficients of P
 $P_{\text{even}}(x^2) = a_0 + a_2 \cdot x^2 + a_4 \cdot x^4 \dots$
- ▶ the coefficients of P_{odd} are the odd-degree coefficients of P
 $P_{\text{odd}}(x^2) = a_1 + a_3 \cdot x^2 + a_5 \cdot x^4 \dots$
- ▶ both P_{even} and P_{odd} have $\lceil n/2 \rceil$ coefficients
- ▶ **problem:** need to evaluate them at n sample points
 $(x_0)^2, \dots, (x_{n-1})^2$

Divide and conquer

Start with conversion from coefficients $\rightarrow$ point-value

OBS: $P(x) = P_{\text{even}}(x^2) + x \cdot P_{\text{odd}}(x^2)$

- ▶ the coefficients of P_{even} are the even-degree coefficients of P
 $P_{\text{even}}(x^2) = a_0 + a_2 \cdot x^2 + a_4 \cdot x^4 \dots$
- ▶ the coefficients of P_{odd} are the odd-degree coefficients of P
 $P_{\text{odd}}(x^2) = a_1 + a_3 \cdot x^2 + a_5 \cdot x^4 \dots$
- ▶ both P_{even} and P_{odd} have $\lceil n/2 \rceil$ coefficients
- ▶ **problem:** need to evaluate them at n sample points
 $(x_0)^2, \dots, (x_{n-1})^2$
- ▶ **unless:** $(x_0)^2, \dots, (x_{n-1})^2$ contains only $n/2$ *distinct* values

Divide and conquer (2)

We want $(x_0)^2, \dots, (x_{n-1})^2$ to contain only $n/2$ *distinct* values

- ▶ use the fact that $x^2 = (-x)^2$
- ▶ one kind of set which satisfies this is if we have half of them be some set of values X and the other is $-X = \{-x | x \in X\}$
- ▶ but we need arbitrarily large such sets defined *recursively*

Divide and conquer (2)

We want $(x_0)^2, \dots, (x_{n-1})^2$ to contain only $n/2$ *distinct* values

- ▶ use the fact that $x^2 = (-x)^2$
- ▶ one kind of set which satisfies this is if we have half of them be some set of values X and the other is $-X = \{-x | x \in X\}$
- ▶ but we need arbitrarily large such sets defined *recursively*
- ▶ start with $n = 1$: use sample set $X_1 = \{1\}$

Divide and conquer (2)

We want $(x_0)^2, \dots, (x_{n-1})^2$ to contain only $n/2$ *distinct* values

- ▶ use the fact that $x^2 = (-x)^2$
- ▶ one kind of set which satisfies this is if we have half of them be some set of values X and the other is $-X = \{-x | x \in X\}$
- ▶ but we need arbitrarily large such sets defined *recursively*
- ▶ start with $n = 1$: use sample set $X_1 = \{1\}$
- ▶ then for $n = 2$: use sample set $X_2 = X_1 \cup \{-1\}$

Divide and conquer (2)

We want $(x_0)^2, \dots, (x_{n-1})^2$ to contain only $n/2$ *distinct* values

- ▶ use the fact that $x^2 = (-x)^2$
- ▶ one kind of set which satisfies this is if we have half of them be some set of values X and the other is $-X = \{-x | x \in X\}$
- ▶ but we need arbitrarily large such sets defined *recursively*
- ▶ start with $n = 1$: use sample set $X_1 = \{1\}$
- ▶ then for $n = 2$: use sample set $X_2 = X_1 \cup \{-1\}$
- ▶ for $n = 4$: use sample set $X_4 = X_2 \cup \{i, -i\}$, where $i^2 = -1$

Divide and conquer (2)

We want $(x_0)^2, \dots, (x_{n-1})^2$ to contain only $n/2$ *distinct* values

- ▶ use the fact that $x^2 = (-x)^2$
- ▶ one kind of set which satisfies this is if we have half of them be some set of values X and the other is $-X = \{-x | x \in X\}$
- ▶ but we need arbitrarily large such sets defined *recursively*
- ▶ start with $n = 1$: use sample set $X_1 = \{1\}$
- ▶ then for $n = 2$: use sample set $X_2 = X_1 \cup \{-1\}$
- ▶ for $n = 4$: use sample set $X_4 = X_2 \cup \{i, -i\}$, where $i^2 = -1$
- ▶ for $n = 8$: use sample set $X_8 = X_4 \cup \{\frac{1+i}{\sqrt{2}}, \frac{1-i}{\sqrt{2}}, \frac{-1+i}{\sqrt{2}}, \frac{-1-i}{\sqrt{2}}\}$
- ▶ ...

The roots of unity

These are called the n^{th} (complex) *roots of unity*

- ▶ they are the roots of the polynomial $x^n = 1$
- ▶ there is a *primitive* root:

$$\omega_n = \cos \frac{2\pi}{n} + i \cdot \sin \frac{2\pi}{n}$$

- ▶ all the other roots are *powers* of the primitive root:
 $(\omega_n)^1, (\omega_n)^2, \dots, (\omega_n)^n$, where

$$(\omega_n)^k = \cos \frac{2\pi \cdot k}{n} + i \cdot \sin \frac{2\pi \cdot k}{n}$$

- ▶ they are spread out evenly on the boundary of a circle and exhibit some very nice properties which we will use

The properties

$$(\omega_n)^k = \cos \frac{2\pi \cdot k}{n} + i \cdot \sin \frac{2\pi \cdot k}{n}$$

- ▶ there are n different roots: $(\omega_n)^k = (\omega_n)^{k \bmod n}$
- ▶ for even n : $(\omega_n)^{k+n/2} = -(\omega_n)^k$
- ▶ $\frac{1}{(\omega_n)^k} = \overline{(\omega_n)^k} = (\overline{\omega_n})^k$, where $\overline{a + b \cdot i} = a - b \cdot i$
- ▶ $\omega_n = (\omega_{kn})^k$

The properties

$$(\omega_n)^k = \cos \frac{2\pi \cdot k}{n} + i \cdot \sin \frac{2\pi \cdot k}{n}$$

- ▶ there are n different roots: $(\omega_n)^k = (\omega_n)^{k \bmod n}$
- ▶ for even n : $(\omega_n)^{k+n/2} = -(\omega_n)^k$
- ▶ $\frac{1}{(\omega_n)^k} = \overline{(\omega_n)^k} = (\overline{\omega_n})^k$, where $\overline{a + b \cdot i} = a - b \cdot i$
- ▶ $\omega_n = (\omega_{kn})^k$
- ▶ these ensure that our divide-and-conquer approach will work

The Fourier Transform

- ▶ for a polynomial p of degree $n - 1$, pick your point-value pairs by evaluating the polynomial at the n^{th} roots of unity
- ▶ the n values you get back are called the *discrete Fourier transform* of the polynomial (or its Fourier coefficients)
- ▶ given a coefficient representation $P[j] = a_j$, you get a vector P^* of n values defined as

$$P^*[j] = p((\omega_n)^j) = \sum_{k=0}^{n-1} P[k] \cdot (\omega_n)^{kj}$$

The Fourier Transform - Divide and Conquer

So would we do a divide and conquer approach?

- ▶ from $P[]$, get $P_{\text{even}}[]$ and $P_{\text{odd}}[]$ (linear time)
- ▶ you want to evaluate P_{even} and P_{odd} at x^2 , where x are the n^{th} roots of unity
- ▶ but x^2 where x are the n^{th} roots of unity = the $n/2^{\text{th}}$ roots of unity, so you can just recurse to get $P_{\text{even}}^*[]$ and $P_{\text{odd}}^*[]$
- ▶ for $j = 0, \dots, n/2 - 1$, we have
$$P^*[j] = P_{\text{even}}^*[j] + (\omega_n)^j \cdot P_{\text{odd}}^*[j]$$
- ▶ for $j = n/2, \dots, n - 1$, we have that $(\omega_n)^j = -(\omega_n)^{j-n/2}$, so
$$P^*[j] = P_{\text{even}}^*[j - n/2] - (\omega_n)^{j-n/2} \cdot P_{\text{odd}}^*[j - n/2]$$

Fast Fourier Transform

```
FFT( $P[0..n-1]$ ):  
  if  $n = 1$   
    return  $P$   
  
  for  $j \leftarrow 0$  to  $n/2 - 1$   
     $U[j] \leftarrow P[2j]$   
     $V[j] \leftarrow P[2j + 1]$   
  
   $U^* \leftarrow \text{FFT}(U[0..n/2 - 1])$   
   $V^* \leftarrow \text{FFT}(V[0..n/2 - 1])$   
  
   $\omega_n \leftarrow \cos(\frac{2\pi}{n}) + i \sin(\frac{2\pi}{n})$   
   $\omega \leftarrow 1$   
  
  for  $j \leftarrow 0$  to  $n/2 - 1$   
     $P^*[j] \leftarrow U^*[j] + \omega \cdot V^*[j]$   
     $P^*[j + n/2] \leftarrow U^*[j] - \omega \cdot V^*[j]$   
     $\omega \leftarrow \omega \cdot \omega_n$   
  
  return  $P^*[0..n-1]$ 
```

- the Fourier transform can be computed in $O(n \log n)$ time by the *Fast Fourier Transform* algorithm (Cooley and Turkey 1965...)

Inverse FT

Now to go from point-value $\rightarrow$ coefficients, our Vandermonde matrix looks like:

$$V = \begin{bmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_n & \omega_n^2 & \omega_n^3 & \cdots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \omega_n^6 & \cdots & \omega_n^{2(n-1)} \\ 1 & \omega_n^3 & \omega_n^6 & \omega_n^9 & \cdots & \omega_n^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \omega_n^{3(n-1)} & \cdots & \omega_n^{(n-1)^2} \end{bmatrix}$$

One can show that: $V^{-1} = \frac{1}{n} \cdot \overline{V}$, where $\overline{V}[j][k] = \overline{V[j][k]} = \overline{\omega_n}^{jk}$

► same algorithm works, just with $\overline{\omega_n}$ instead of ω_n

Inverse FFT

```
INVERSEFFT( $P^*[0..n-1]$ ):  
  if  $n = 1$   
    return  $P$   
  for  $j \leftarrow 0$  to  $n/2 - 1$   
     $U^*[j] \leftarrow P^*[2j]$   
     $V^*[j] \leftarrow P^*[2j + 1]$   
   $U \leftarrow \text{INVERSEFFT}(U[0..n/2 - 1])$   
   $V \leftarrow \text{INVERSEFFT}(V[0..n/2 - 1])$   
   $\overline{\omega_n} \leftarrow \cos(\frac{2\pi}{n}) - i \sin(\frac{2\pi}{n})$   
   $\omega \leftarrow 1$   
  for  $j \leftarrow 0$  to  $n/2 - 1$   
     $P[j] \leftarrow 2(U[j] + \omega \cdot V[j])$   
     $P[j + n/2] \leftarrow 2(U[j] - \omega \cdot V[j])$   
     $\omega \leftarrow \omega \cdot \overline{\omega_n}$   
  return  $P[0..n-1]$ 
```

Back to polynomial multiplication

FFTMULTIPLY($P[0..n-1], Q[0..m-1]$):

$\ell \leftarrow \lceil \lg(n+m) \rceil$

for $j \leftarrow n$ to $2^\ell - 1$

$P[j] \leftarrow 0$

for $j \leftarrow m$ to $2^\ell - 1$

$Q[j] \leftarrow 0$

$P^* \leftarrow FFT(P)$

$Q^* \leftarrow FFT(Q)$

for $j \leftarrow 0$ to $2^\ell - 1$

$R^*[j] \leftarrow P^*[j] \cdot Q^*[j]$

return INVERSEFFT(R^*)

Claim 1.

Polynomial multiplication can be done in $O(n \log n)$ arithmetic operations using FFT.

Interlude

Why are these operations useful?

1. Describe an algorithm that determines whether a given set of n integers contains two elements whose sum is zero, in $O(n \log n)$ time.
2. Describe an algorithm that determines whether a given set of n integers contains three elements whose sum is zero, in $O(n^2)$ time.
3. Now suppose the input set X contains only integers between $-10000n$ and $10000n$. Describe an algorithm that determines whether X contains three elements whose sum is zero, in $O(n \log n)$ time.

Thank you!